PREVIO 2 DESARROLLO DE CONTENIDOS DIGITALES Y MULTIMEDIALES

MOMENTO 2 DE SU PROYECTO DE VIDEOJUEGO EN UNITY

ACTIVIDAD	**ENTREGABLES**	**FECHA DE ENTREGA Y DESARROLLO DEL SERIUS GAME EN UNITY**
Actividades: • Integración de personajes y escenarios en Unity. • Programación de mecánicas básicas: movimiento, interacción, recolección, diálogos. • Implementación de HUD y menú funcional. (El **HUD** (Head-Up Display) en un *serious game* es la **interfaz visual que muestra información clave al jugador durante la partida**, sin interrumpir la experiencia de juego. Es como el "panel de control" que siempre está visible en pantalla). •	**Entregables:** El estudiante entrega cinco productos livianos; no se solicita el proyecto completo de Unity, que pesa gigas y retrasa la revisión. Todo se entrega como enlaces o archivos pequeños en un único formulario o actividad de la plataforma. 1. Informe técnico (PDF) 2. Evidencias gráficas 3. Video explicativo 4. Build jugable 5. Repositorio de código fuente en GitHub	AVANCE 2 (VERSIÓN JUGABLE, VIDEO DEMOSTRATIVO) Del 13 al 18 de octubre de 2026 PREVIO 2 – PRESENTAR PRODUCTOS ENTREGABLES, TENIENDO PRESENTE RÚBRICA DE EVALUACIÓN: Del 19 al 23 de octubre de 2026, se realizan las sustentaciones – previo 2 (60%), presentaciones y entrega de productos así: B193: 19 Y 21 DE OCTUBRE DE 2026 B192: 20 DE OCTUBRE DE 2026 A LAS 7:30 A.M. Y 12 M.

Momento 2 - Desarrollo funcional y mecánicas básicas de un Serious Game en Unity (Proyecto Unity del semestre)

Contexto

Una organización desea crear un videojuego educativo - Serious Game en Unity para enseñar un tema de interés social, ambiental, empresarial o académico.

Usted ha sido contratado como desarrollador junior de videojuegos y debe construir un prototipo funcional utilizando Unity, aplicando las mecánicas básicas de movimiento, interacción, recolección de elementos, diálogos y una interfaz de usuario que permita al jugador comprender el estado del juego.

Al finalizar, deberá demostrar que todas las funcionalidades funcionan correctamente mediante pruebas y evidencias.

TALLER PRÁCTICO 3: DIÁLOGOS CON NPC, INTERACCIÓN CON OBJETOS, RECOLECCIÓN Y HUD

Previo 2: Desarrollo Funcional y Mecánicas Básicas de un Serious Game en Unity 2D

1.OBJETIVOS Y ALINEACIÓN CON EL PREVIO 2

Este taller guía la implementación paso a paso de los requisitos evaluables del Previo 2 para tu Serious Game. Al finalizar esta sesión, habrás construido:

- **1 NPC con diálogo interactivo**: conversable con la tecla **E**, con textos educativos editables desde el Inspector.
- **2 Objetos interactivos**: con indicador flotante ("Presiona E para interactuar") que activan acciones educativas (leer libros/carteles, activar mecanismos).
- **2 Objetos coleccionables**: que se recogen mediante triggers, desaparecen y suman al progreso sin duplicarse.
- **HUD (Head-Up Display) funcional**: con contador de progreso (ej. "3/5"), objetivo actual visible y mini-inventario con iconos.
- **Adaptaciones opcionales**: tutorial de salto para plataformas, colocación de fondos y adaptación para vistas cenitales (Top-Down).

Rúbrica de Evaluación del Previo 2 (100%)

- **Escenario y personajes**: 10%
- **Movimiento**: 15%
- Interacción con objetos: 15%
- **Sistema de recolección**: 15%
- **Sistema de diálogos**: 15%
- **HUD funcional**: 10%
- **Menú funcional**: 10%
- **Pruebas funcionales**: 5%
- **Documentación e informe técnico**: 5%

2. MÓDULOS COMPLEMENTARIOS: AJUSTES PREVIOS SEGÚN EL TIPO DE JUEGO

A. Tutorial Opcional: Mecánica de Salto (Para Juegos de Plataformas 2D)

Si tu Serious Game es de vista lateral (estilo Hollow Knight o Mario) y requiere salto, ajusta los tilemaps para hacer plataformas flotantes donde usar esta mecánica:

1. En el GameObject del suelo (`Tilemap`), asigna la Layer `Ground` (crea una nueva capa llamada Ground en el Inspector > Layer).
2. Agrega este bloque de salto al script `PlayerController.cs`:

□Header("Configuración de Salto (Opcional Plataformas)")]

□SerializeField] private float jumpForce = 12f;

□SerializeField] private Transform groundCheck;

□SerializeField] private float checkRadius = 0.2f;

□SerializeField] private LayerMask groundLayer;

private bool isGrounded;

// Dentro de Update():

isGrounded = Physics2D.OverlapCircle(groundCheck.position, checkRadius, groundLayer);

if (Input.GetButtonDown("Jump") && isGrounded)

{

rb.velocity = new Vector2(rb.velocity.x, jumpForce);

3

Montaje: Crea un GameObject vacío como hijo del Player ubicado exactamente bajo sus pies y asígnalo al campo **Ground Check**.

B. Colocación de Fondo Decorativo en la Escena de Juego

1. Arrastra tu imagen de fondo a la jerarquía de `GameScene`.

2. En su `SpriteRenderer`, cambia la propiedad **Sorting Layer** a `Background` (la capa más atrasada).
3. Escala el fondo para cubrir el área del nivel o duplícalo horizontalmente para abarcar toda la plataforma.

3. FASE 1: IMPLEMENTACIÓN DEL NPC Y SISTEMA DE DIÁLOGOS

A. Dónde conseguir Sprites para NPCs

- En **itch.io Game Assets**, busca bajo las etiquetas `npc sprite sheet`, `villager pixel art` o `dialogue character`.
- Asegúrate de recortar el spritesheet en modo `Multiple` y asignar el primer sprite a la escena.

B. Montaje del NPC en la Jerarquía

1. Arrastra el sprite del NPC a la escena y nómbralo `NPC_Guia`.
2. Asigna su **Sorting Layer** en `Characters` y `Order in Layer` en `0` o `1`.
3. Agrega un componente **BoxCollider2D** o **CircleCollider2D** y marca activada la casilla **Is Trigger**.
    - Ajusta el radio o tamaño del collider para delimitar el "área de conversación" (cuando el jugador entra a esta área, puede hablar con él).
4. **Globo indicador visual**: Agrega un objeto hijo al NPC con un sprite pequeño de tecla **E** o bocadillo de texto que se encienda al acercarse.

C. Creación del Panel de Diálogo en el Canvas

En el `Canvas` de `GameScene`, crea:

1. Un `UI > Panel` llamado `DialoguePanel` anclado en la parte inferior de la pantalla.
2. Un `UI > Text - TextMeshPro` para el nombre del personaje (`NPCNameText`).
3. Un `UI > Text - TextMeshPro` para el contenido del mensaje (`DialogueContentText`).
4. Un texto indicador pequeño en la esquina inferior: *"Presiona E o Espacio para continuar..."*.
5. Desactiva la casilla activa de `DialoguePanel` en el Inspector para que comience oculto.

D. Script del Sistema de Diálogo (`NPCDialogue.cs`)

Crea este script en `Assets/Scripts/` y adjúntalo al GameObject `NPC_Guia`:using UnityEngine;

```
using TMPro;

public class NPCDialogue : MonoBehaviour

{

    □Header("Configuración del NPC")]

    □SerializeField] private string npcName = "Sabio Medieval";

    □TextArea(2, 5)□

    □SerializeField] private string[] dialogueLines;

    □Header("Referencias de Interfaz (UI)")□

    □SerializeField] private GameObject dialoguePanel;

    □SerializeField] private TMP_Text nameText;

    □SerializeField] private TMP_Text contentText;

    □SerializeField] private GameObject interactionPrompt; // Icono flotante "E"

    private bool isPlayerInRange = false;

    private bool isDialogueActive = false;

    private int currentLineIndex = 0;

    private void Start()

    {

        if (dialoguePanel !□ null) dialoguePanel.SetActive(false);

        if (interactionPrompt !□ null) interactionPrompt.SetActive(false);

    }

    private void Update()
```

```
{
    if (isPlayerInRange && Input.GetKeyDown(KeyCode.E))
    {
        if (!isDialogueActive)
        {
            StartDialogue();
        3
        else
        {
            NextDialogueLine();
        3
    3
3
private void StartDialogue()
{
    isDialogueActive = true;
    currentLineIndex = 0;
    dialoguePanel.SetActive(true);
    if (interactionPrompt !□ null) interactionPrompt.SetActive(false);
    nameText.text = npcName;
    contentText.text = dialogueLines[currentLineIndex];
3
```

```
private void NextDialogueLine()
{
    currentLineIndex++;
    if (currentLineIndex < dialogueLines.Length)
    {
        contentText.text = dialogueLines[currentLineIndex];
    }
    else
    {
        EndDialogue();
    }
}
private void EndDialogue()
{
    isDialogueActive = false;
    dialoguePanel.SetActive(false);
    if (interactionPrompt != null) interactionPrompt.SetActive(true);
}
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Player"))
```

```
    {
        isPlayerInRange = true;
        if (!isDialogueActive && interactionPrompt !□ null)
            interactionPrompt.SetActive(true);
    }
}
private void OnTriggerExit2D(Collider2D collision)
{
    if (collision.CompareTag("Player"))
    {
        isPlayerInRange = false;
        EndDialogue();
        if (interactionPrompt !□ null) interactionPrompt.SetActive(false);
    }
}
```

En la ventana **Hierarchy**, haz clic sobre el GameObject de tu NPC (por ejemplo, `NPC_Guia`).
Mira hacia la ventana **Inspector** a la derecha, en el componente **NPCDialogue (Script)**. Verás una sección llamada **Dialogue Lines**:

- Si dice `Size: 0`, cámbialo a la cantidad de páginas o frases que dirá el NPC (por ejemplo, ponle `3`).
- Al pulsar *Enter*, se desplegarán inmediatamente los campos:
    - **Element 0**: Escribes la primera frase (ej: *"¡Saludos, noble viajero!"*).
    - **Element 1**: Escribes la segunda frase (ej: *"El castillo ha perdido 5 manuscritos históricos..."*).

- **Element 2**: Escribes la tercera frase (ej: *"Tráelos de vuelta para completar tu lección"*)

opcional: ¿Cómo hacer un diálogo interactivo compartido (Jugador ↔ NPC)?

Para que el jugador y el NPC tengan una conversación real y el nombre superior cambie dinámicamente según quién hable (ej. *"Caballero"* vs *"Sabio"*), en lugar de tener solo una lista de textos planos, creamos una **estructura de diálogo (DialogueLine)** que guarde **quién habla** y **qué dice**.

El Script Mejorado: `NPCDialogueAvanzado.cs`

Reemplaza o crea este script en tu carpeta `Assets/Scripts/`:

```
using UnityEngine;
using TMPro;

// Estructura que une el Nombre del Hablante con su Texto
□System.Serializable]
public struct DialogueLine
{
    □Tooltip("Escribe aquí el nombre: ej. 'Caballero (Tú)' o 'Sabio Medieval'")]
    public string speakerName;

    □Tooltip("Color opcional para distinguir el nombre de quien habla")]
    public Color nameColor;

    □TextArea(2, 5)□
    public string text;
3

public class NPCDialogueAvanzado : MonoBehaviour
{
    □Header("Secuencia de Conversación")]
    □SerializeField] private DialogueLine[] conversation;

    □Header("Referencias de Interfaz (UI)")□
    □SerializeField] private GameObject dialoguePanel;
    □SerializeField] private TMP_Text nameText;
    □SerializeField] private TMP_Text contentText;
    □SerializeField] private GameObject interactionPrompt; // Icono flotante "Presiona E"

    private bool isPlayerInRange = false;
```

```
private bool isDialogueActive = false;
private int currentLineIndex = 0;

private void Start()
{
    if (dialoguePanel !□ null) dialoguePanel.SetActive(false);
    if (interactionPrompt !□ null) interactionPrompt.SetActive(false);
3

private void Update()
{
    if (isPlayerInRange && Input.GetKeyDown(KeyCode.E))
    {
        if (!isDialogueActive)
        {
            StartDialogue();
        3
        else
        {
            NextDialogueLine();
        3
    3
3

private void StartDialogue()
{
    if (conversation.Length == 0) return;

    isDialogueActive = true;
    currentLineIndex = 0;
    dialoguePanel.SetActive(true);
    if (interactionPrompt !□ null) interactionPrompt.SetActive(false);

    DisplayCurrentLine();
3

private void NextDialogueLine()
{
    currentLineIndex++;
    if (currentLineIndex < conversation.Length)
    {
        DisplayCurrentLine();
```

```
    }
    else
    {
        EndDialogue();
    }
}

private void DisplayCurrentLine()
{
    // 1. Actualiza el nombre del hablante en el cuadro superior
    nameText.text = conversation[currentLineIndex].speakerName;

    // 2. Si definiste un color para el nombre, lo aplica (ej: dorado para el jugador)
    if (conversation[currentLineIndex].nameColor.a > 0f)
    {
        nameText.color = conversation[currentLineIndex].nameColor;
    }

    // 3. Actualiza el texto de lo que está diciendo
    contentText.text = conversation[currentLineIndex].text;
}

private void EndDialogue()
{
    isDialogueActive = false;
    dialoguePanel.SetActive(false);
    if (interactionPrompt != null) interactionPrompt.SetActive(true);
}

private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Player"))
    {
        isPlayerInRange = true;
        if (!isDialogueActive && interactionPrompt != null)
            interactionPrompt.SetActive(true);
    }
}

private void OnTriggerExit2D(Collider2D collision)
{
    if (collision.CompareTag("Player"))
```

```
    {
        isPlayerInRange = false;
        EndDialogue();
        if (interactionPrompt != null) interactionPrompt.SetActive(false);
    }
}
}
```

En **Conversation > Size**, defines la cantidad de intervenciones (por ejemplo, `4`). Cada elemento te pedirá de forma limpia:

- Element 0:
  - *Speaker Name*: `Sabio Medieval` (Color: Blanco)
  - *Text*: *"¡Bienvenido al monasterio, joven caballero!"*
- Element 1:
  - *Speaker Name*: `Caballero (Tú)` (Color: Amarillo / Dorado)
  - *Text*: *"Saludos. Vengo en busca de los textos históricos perdidos."*
- Element 2:
  - *Speaker Name*: `Sabio Medieval` (Color: Blanco)
  - *Text*: *"Están dispersos por el bosque. Encuentra los 5 manuscritos."*
- Element 3:
  - *Speaker Name*: `Caballero (Tú)` (Color: Amarillo / Dorado)
  - *Text*: *"Entendido, los reuniré de inmediato."*

4. FASE 2: INTERACCIÓN CON OBJETOS EDUCATIVOS (MÍNIMO 2 OBJETOS CON TECLA E)

El Previo 2 exige 2 objetos interactivos con mensajes al acercarse y activación mediante la tecla **E** (ej. un cofre histórico, una puerta, un cartel informativo o un artefacto antiguo).

Script Modular de Objeto Interactivo (InteractableObject.cs)

Crea el script en `Assets/Scripts/` y adjúntalo a tus objetos interactivos:using UnityEngine;

using TMPro;

public class InteractableObject : MonoBehaviour

{

```
    □Header("Mensaje  Flotante")]

    □SerializeField] private GameObject promptFloatingText; // Texto "Presiona E para
leer"

    □Header("Contenido  Educativo")]

    □SerializeField] private GameObject infoDisplayPanel;

    □SerializeField] private TMP_Text infoTextComponent;

    □TextArea(3, 6)□

    □SerializeField] private string educationalContent = "Dato Histórico: En la Edad
Media, los manuscritos eran iluminados a mano por monjes copistas.";

    private bool isPlayerNearby = false;

    private bool isPanelOpen = false;

    private void Start()

    {

        if (promptFloatingText !□ null) promptFloatingText.SetActive(false);

        if (infoDisplayPanel !□ null) infoDisplayPanel.SetActive(false);

    3

    private void Update()

    {

        if (isPlayerNearby && Input.GetKeyDown(KeyCode.E))

        {

            ToggleInteraction();

        3

    3
```

```
private void ToggleInteraction()
{
    isPanelOpen = !isPanelOpen;
    if (infoDisplayPanel !□ null) infoDisplayPanel.SetActive(isPanelOpen);
    if (infoTextComponent !□ null) infoTextComponent.text = educationalContent;
    if (promptFloatingText !□ null) promptFloatingText.SetActive(!isPanelOpen);
}
private void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        isPlayerNearby = true;
        if (promptFloatingText !□ null && !isPanelOpen)
            promptFloatingText.SetActive(true);
    }
}
private void OnTriggerExit2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        isPlayerNearby = false;
```

```
        isPanelOpen = false;
        if (promptFloatingText !□ null) promptFloatingText.SetActive(false);
        if (infoDisplayPanel !□ null) infoDisplayPanel.SetActive(false);
    3
3
3
```

Requisitos del objeto:

- Debe tener un `Collider2D` con **`Is Trigger = true`**.
- El `Player` debe tener asignado el Tag oficial **`Player`** en el Inspector.

5. FASE 3: SISTEMA DE RECOLECCIÓN (MÍNIMO 5 OBJETOS COLECCIONABLES)

Para el caso guía, utilizaremos 5 pergaminos históricos distribuidos a lo largo del nivel.

Script de Objeto Coleccionable (`CollectibleItem.cs`)

```
using UnityEngine;
public class CollectibleItem : MonoBehaviour
{
    □Header("Identificación")]
    □SerializeField] private string itemName = "Pergamino Histórico";
    □SerializeField] private int itemValue = 1;
    private bool isCollected = false;
    private void OnTriggerEnter2D(Collider2D collision)
    {
        // Validación rigurosa: solo el Player y solo una vez (Prueba P3)
```

```
    if (collision.CompareTag("Player") && !isCollected)
    {
        isCollected = true;

        // Notificar al gestor del HUD
        if (GameHUDManager.Instance != null)
        {
            GameHUDManager.Instance.AddCollectible(itemValue);
        }
        // Desaparecer el objeto de la escena
        Destroy(gameObject);
    }
}
```

6. FASE 4: IMPLEMENTACIÓN DEL HUD (HEAD-UP DISPLAY)

El HUD es la interfaz visible durante toda la partida que informa al jugador sobre sus metas y avances.

A. Elementos del HUD en el Canvas

1. **Panel Superior de Misión**:
    - `UI > Text - TextMeshPro` con el texto de objetivo: *"Misión: Recolecta las 5 Notas Medievales"*.
2. Contador de Coleccionables:
    - `UI > Text - TextMeshPro` que muestre: *"Notas: 0/5"*.
    - *(Opcional)* Un icono visual del pergamino o residuo junto al contador.
3. Panel de Misión Cumplida:

- Un panel modal que se activa automáticamente al llegar a 5/5 con el mensaje: *"¡Felicidades! Has completado el objetivo pedagógico. Habla con el sabio para fiinalizar"*.

B. Script del Administrador del HUD (`GameHUDManager.cs`)

```
using UnityEngine;

using TMPro;

public class GameHUDManager : MonoBehaviour

{

    public static GameHUDManager Instance { get; private set; 3

    □Header("Referencias de Texto HUD")]

    □SerializeField] private TMP_Text counterText;

    □SerializeField] private TMP_Text objectiveText;

    □SerializeField] private GameObject missionCompletePanel;

    □Header("Configuración de Meta")]

    □SerializeField] private int totalRequired = 5;

    private int currentCollected = 0;

    private void Awake()

    {

        if (Instance == null) Instance = this;

        else Destroy(gameObject);

    3

    private void Start()

    {
```

```
    UpdateHUDVisuals();
    if (missionCompletePanel != null) missionCompletePanel.SetActive(false);
}
public void AddCollectible(int amount)
{
    currentCollected += amount;
    UpdateHUDVisuals();
    if (currentCollected >= totalRequired)
    {
        OnMissionCompleted();
    }
}
 private void UpdateHUDVisuals()
{
    if (counterText != null)
    {
        counterText.text = $"Recolectados: {currentCollected3 / {totalRequired3";
    }
}
private void OnMissionCompleted()
{
    if (missionCompletePanel != null)
```

```
        {
            missionCompletePanel.SetActive(true);
        }
    }
}
```

7. FASE 5: TABLA DE PRUEBAS FUNCIONALES (VALIDACIÓN OBLIGATORIA PREVIO 2)

Para la sustentación y el informe técnico, debes validar y registrar la siguiente matriz de pruebas funcionales:

ID	Caso de Prueba	Procedimiento de Prueba	Resultado Esperado
P1	Movimiento contra muro	Caminar continuamente hacia un collider sólido	El personaje frena en seco y no lo atraviesa
P2	Pulsar tecla E lejos	Presionar E sin estar dentro del trigger de un objeto	No ocurre ninguna acción ni se abren paneles
P3	Coleccionable único	Recolectar un ítem y pasar de nuevo por su posición	El ítem desaparece y solo se contabiliza 1 vez
P4	Diálogo completo con NPC	Presionar E sucesivamente frente al NPC	Avanza línea por línea y se cierra al terminar
P5	Diálogos duplicados	Intentar presionar E para abrir otro diálogo mientras uno está activo	El sistema no solapa ni duplica interfaces
P6	Retroalimentación educativa	Interactuar con un objeto informativo	Muestra el texto pedagógico con claridad y legibilidad
P7	Reinicio del juego	Cargar la escena nuevamente	Contadores, jugador e ítems vuelven a su estado inicial

ID	Caso de Prueba	Procedimiento de Prueba	Resultado Esperado
P8	Ejecución de Build	Probar el ejecutable comprimido o versión WebGL	Corre fluidamente sin caídas de frame ni errores de consola
P9	Actualización de HUD	Recolectar cualquier ítem en la escena	El texto del contador en pantalla se actualiza en tiempo real
P10	Completar misión	Recolectar los 5 coleccionables obligatorios	El sistema activa el mensaje o panel de meta cumplida

8. GUÍA DE ENTREGA Y CHECKLIST FINAL

Para obtener la calificación máxima según la rúbrica del Previo 2:

1. **Informe Técnico (PDF)**: Portada, objetivos del Serious Game, descripción de mecánicas, mínimo 10 capturas de pantalla, tabla de pruebas P1 a P10 diligenciada y conclusiones.
2. **Video Explicativo (5 a 10 minutos)**: Grabación de pantalla (puede ser usando la tecla windows+G y empezar a grabar la pantalla) con voz mostrando el menú funcional, movimiento con colisiones, interacción con NPC, lectura de los 2 objetos interactivos, recolección de los 5 ítems y actualización del HUD.
3. **Build Jugable**: Archivo ZIP ejecutable para Windows/Mac o enlace WebGL publicado en itch.io.
4. **Repositorio GitHub**: Enlace al repositorio público con los scripts C#, archivo `README.md` explicativo y archivo `.gitignore` configurado para Unity (excluyendo carpetas pesadas como `Library/` y `Temp/`).